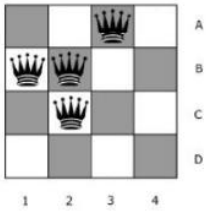
BÀI TẬP CSP VÀ GAME TRONG AI

Họ và tên	Đỗ Xuân Triệu
MSSV / Lớp	3124720072

Bài 1. Bài toán 4-Queens và Hill Climbing

```
function 4-QUEENS-HILL-CLIMBING(problem)
    current_state = problem.INITIAL-STATE
    loop do
        neighbor = state with lowest number of conflicts between queens obtained by
                   making one legal chess move from current_state
        if neighbor.num_conflicts >= current_state.num_conflicts
            return current_state
        current_state = neighbor
```



Hình 1. Mã giả Hill Climbing và trạng thái khởi đầu của bài toán 4-Queens

a) Hãy ghi trạng thái của từng quân hậu sau một vòng lặp của thuật toán. Một vòng lặp tương đương với việc di chuyển một quân hậu.

- Trạng thái ban đầu (theo cột 1 đến 4): Q1 ở hàng 2 (B1), Q2 ở hàng 3 (C2), Q3 ở hàng 1 (A3), Q4 ở hàng 2 (B4).

- Số conflicts ban đầu là 3.

- Sau một vòng lặp, thuật toán thử di chuyển từng quân hậu để tìm trạng thái láng giềng có số lỗi nhỏ nhất. Trạng thái tốt nhất đạt được khi di chuyển quân hậu ở cột 4 (Q4) từ hàng 2 sang hàng 4 (D4).

- Trạng thái sau một vòng lặp: Quân hậu 1 ở B1, Quân hậu 2 ở C2, Quân hậu 3 ở A3, Quân hậu 4 ở D4. (Số conflicts giảm xuống còn 1).

b) Thuật toán hill-climbing ở trên có dừng với trạng thái khởi đầu đã cho hay không? Nếu có, trạng thái cuối thuộc loại nào? Chọn tất cả phương án phù hợp.

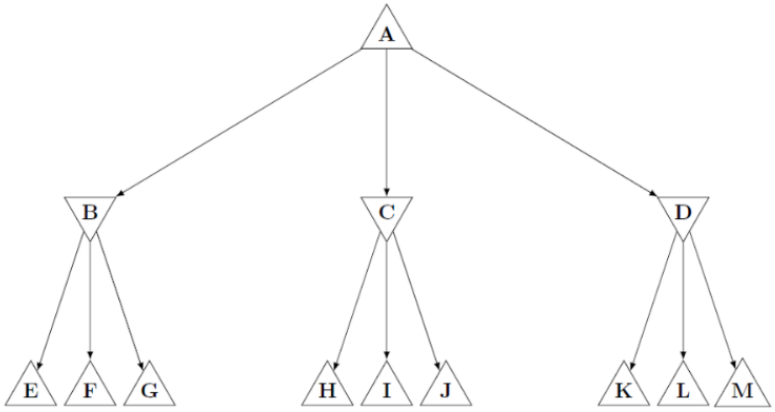
- Có dừng. Sau bước di chuyển đầu tiên, trạng thái đạt được có số lỗi là 1. Mọi láng giềng tiếp theo đều có số lỗi >= 1, nên điều kiện neighbor.num_conflicts >= current_state.num_conflicts được thỏa mãn, thuật toán trả về trạng thái hiện tại

- Các phương án phù hợp:

  B) Cực trị cục bộ (local optimum)

  D) Vai bằng (shoulder)

Bài 2. Alpha-Beta Pruning kết hợp Iterative Deepening



Evaluation Function	
State, s	$f(s)$
B	4
C	14
D	19
E	5
F	7
G	8
H	17
I	16
J	15
K	23
L	20
M	26

Hình 2. Cây trò chơi và bảng giá trị hàm đánh giá

a) Thứ tự các cây con sau khi sắp xếp là:

D -> C -> B

b) Với thứ tự mới, hãy liệt kê các nút KHÔNG được thăm do bị cắt tỉa. Các nút cháu trong mỗi cây con vẫn được xét từ trái sang phải.
- Duyệt D(MIN): K(23), L(20), M(26) -> Trả về 20 cho A(MAX). alpha = 20.
- Duyệt C(MIN): H(17) -> beta của C = 17 <= alpha (20) -> Cắt tỉa I, J.
- Duyệt B(MIN): E(5) -> beta của B = 5 <= alpha (20) -> Cắt tỉa F, G.

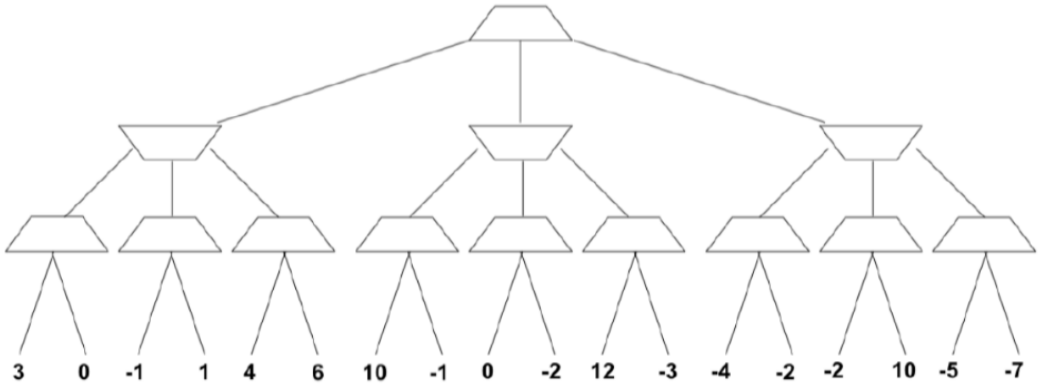
=> Các nút bị cắt tỉa là: I, J, F, G.

c) Alpha-Beta Pruning chuẩn đảm bảo cho cùng nước đi với Minimax đầy đủ. Phương pháp iterative deepening kết hợp pruning như mô tả ở trên có luôn đảm bảo trả về cùng nước đi với Minimax trên toàn bộ cây hay không? Giải thích ngắn gọn.

Có

Alpha-Beta Pruning luôn đảm bảo kết quả chính xác như Minimax. Việc đổi thứ tự duyệt ưu tiên nhánh có tiềm năng tốt hơn chỉ giúp tăng hiệu quả cắt tỉa sớm các nhánh vô ích, chứ không làm thay đổi giá trị Minimax cuối cùng của cây

Bài 3. Adversarial Search - Alpha-Beta Pruning



Hình 3. Cây trò chơi dùng cho Alpha-Beta Pruning

Yêu cầu:

- Ghi giá trị alpha, beta...

- Ghi giá trị được trả về...

- Đánh dấu X lên các nhánh bị cắt tỉa.

- Cho biết giá trị Minimax tại nút gốc và nước đi tối ưu.

- Giá trị Minimax tại nút gốc: 1

- Nước đi tối ưu của MAX: Chọn nhánh thứ 2 từ trái sang

- Các nhánh bị đánh dấu X:

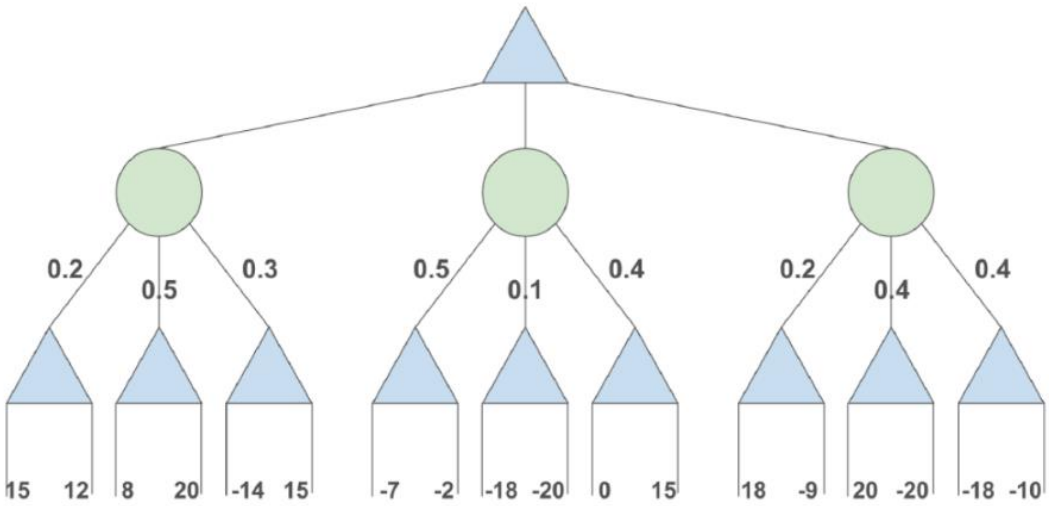
  + Tại cây con thứ 3: Cắt nhánh đến lá (0).

  + Tại cây con thứ 4: Cắt nhánh đến lá (12) và (-3).

  + Tại cây con thứ 5: Cắt nhánh đến hai lá (-2).

  + Tại cây con thứ 6: Cắt nhánh đến lá (-7).

Bài 4. Expectimax và Chance Nodes



a) Duyệt các nút con từ trái sang phải. Hãy tính giá trị được trả về tại từng nút MAX, từng chance node và tại nút gốc.

- Nút Chance 1 (trái):

0.2*max(15,12) + 0.5*max(8,20) + 0.3*max(-14,15)

= 0.2*15 + 0.5*20 + 0.3*15 = 3 + 10 + 4.5 = **17.5**

- Nút Chance 2 (giữa):

0.5*max(-7,-2) + 0.1*max(-18,-20) + 0.4*max(0,15)

= 0.5*(-2) + 0.1*(-18) + 0.4*15 = -1 - 1.8 + 6 = **3.2**

- Nút Chance 3 (phải):

0.2*max(18,-9) + 0.4*max(20,-20) + 0.4*max(-18,-10)

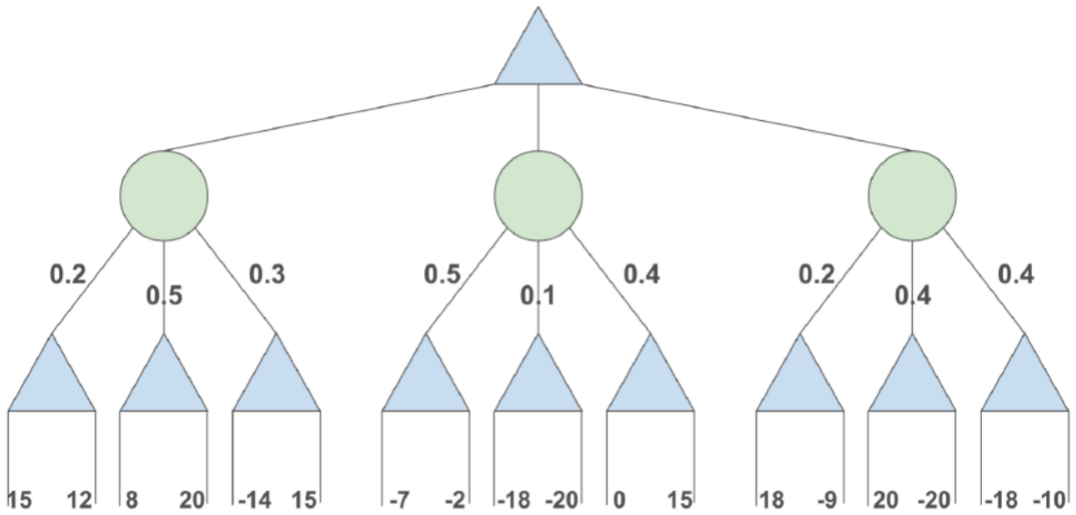
= 0.2*18 + 0.4*20 + 0.4*(-10) = 3.6 + 8 - 4 = **7.6**

- Nút gốc MAX = max(17.5, 3.2, 7.6) = 17.5

b) Nếu chưa biết thêm giới hạn nào về giá trị lá, những nút nào trong cây trên có thể được cắt tỉa mà vẫn đảm bảo kết quả chính xác? Giải thích.

Không có nhánh nào được cắt tỉa. Thuật toán Expectimax thông thường không thể cắt tỉa nhánh nếu không biết trước cận trên (upper bound) của các nút lá. Lý do là vì một xác suất rất nhỏ nếu nhân với một giá trị vô cùng lớn ở nhánh chưa khám phá có thể thay đổi hoàn toàn giá trị kỳ vọng.

c) Giả sử mỗi nút lá đều có giá trị nằm trong đoạn [-20, 20]. Hãy giải lại cây và cắt tỉa các nhánh không cần thiết; vẫn duyệt từ trái sang phải.



Hình 5. Cây Expectimax dùng cho câu 4(c)

- Các nhánh bị cắt tỉa: Nhánh thứ 2 và nhánh thứ 3 của nút Chance 2 (chứa các lá -18, -20, 0, 15).

- Giải thích: Nút Chance 1 trả về 17.5. Tại nút Chance 2, sau khi duyệt nhánh đầu (xác suất 0.5, MAX=-2), giá trị kỳ vọng tạm thời là -1. Dù hai nhánh còn lại có đạt giá trị cực đại là 20, cận trên lớn nhất của Chance 2 chỉ là: -1 + 0.1*20 + 0.4*20 = 9. Vì 9 < 17.5, Chance 2 không thể vượt qua Chance 1, nên ta có thể ngừng duyệt các nhánh còn lại của nó.

Bài 5. Câu hỏi thảo luận về CSP

a) Forward Checking khác AC-3 ở điểm nào? So sánh phạm vi lan truyền ràng buộc, thời điểm thực hiện và chi phí tính toán.

- Phạm vi lan truyền: FC chỉ lan truyền ràng buộc đúng một bước từ biến vừa gán sang các biến kề chưa gán để loại bỏ các giá trị vi phạm. AC-3 lan truyền đệ quy trên toàn bộ đồ thị; nếu miền của một biến bị thay đổi, nó sẽ tiếp tục kiểm tra lại các ràng buộc liên quan để duy trì nhất quán toàn cục.

- Thời điểm: FC được thực thi ngay trong quá trình tìm kiếm (backtracking) sau mỗi phép gán. AC-3 thường dùng làm tiền xử lý trước khi tìm kiếm, hoặc kết hợp bên trong thuật toán tìm kiếm (như thuật toán MAC).

- Chi phí tính toán: FC tốn ít chi phí tính toán hơn tại mỗi bước nhưng bù lại có thể phát hiện mâu thuẫn muộn. AC-3 tốn kém tài nguyên tính toán hơn tại mỗi lần gọi nhưng giúp thu hẹp đáng kể không gian tìm kiếm.

b) Vì sao các heuristic sau thường được dùng trong bài toán CSP?

- i) Minimum Remaining Values (MRV): Chọn biến có ít giá trị khả dĩ nhất. Dựa trên nguyên lý "Fail-first", MRV giúp phát hiện sớm các nhánh sẽ dẫn tới bế tắc để quay lui sớm, qua đó cắt giảm tối đa nhánh tìm kiếm vô ích.

- ii) Least Constraining Value (LCV): Chọn giá trị loại đi ít lựa chọn nhất của các biến xung quanh. Dựa trên nguyên lý "Succeed-first", LCV tối đa hóa cơ hội gán thành công các biến tiếp theo, giúp tìm ra một nghiệm nhanh nhất mà không cần quay lui nhiều.

Bài 6. CSP - Điều phối không lưu

a) Hãy mô hình hóa bài toán dưới dạng CSP: xác định biến, miền giá trị và các ràng buộc đơn/nguyên và nhị phân. Viết ràng buộc bằng ký hiệu toán học hoặc logic.

- Biến: A, B, C, D, E.

- Miền giá trị cho mỗi biến: Tập hợp các cặp (Đường băng, Khung giờ), D_i = {(R, T) | R thuộc {Quốc Tế, Nội Địa}, T thuộc {1, 2, 3, 4}}.

- Ràng buộc unary:

  + B = (R_B, 1) (B hạ cánh khung giờ 1)

  + D = (R_D, T_D) với T_D >= 3

  + A = (R_A, T_A) với T_A <= 2

- Ràng buộc binary:

  + T_D < T_C (D hạ cánh trước C cất cánh)

  + Alldiff(A, B, C, D, E) (không có hai máy bay nào trùng đường băng và trùng giờ)

Bổ sung hai ràng buộc sau:

• A, B, C là các chuyến bay quốc tế và chỉ dùng đường băng quốc tế.

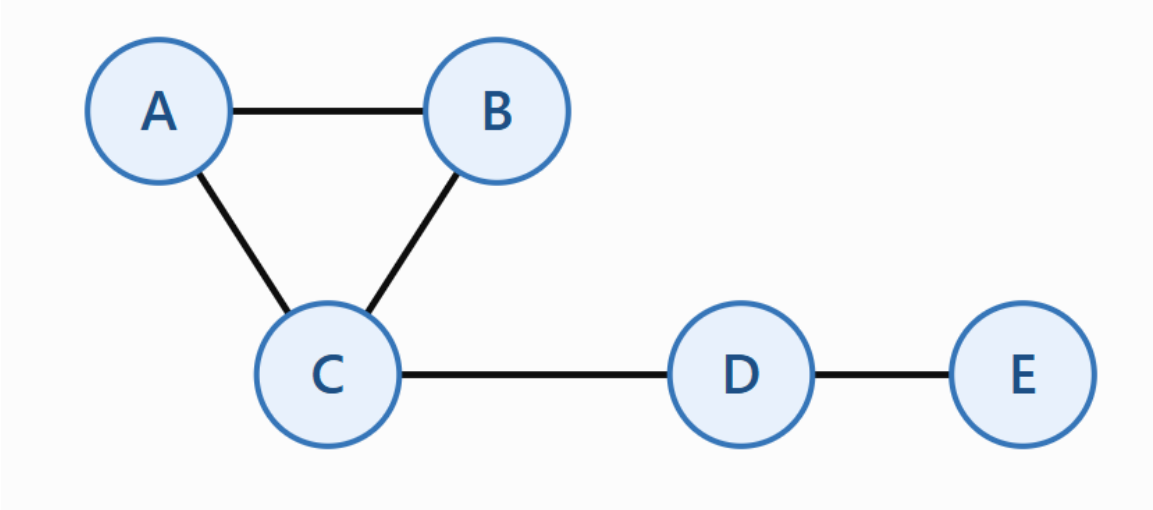
• D, E là các chuyến bay nội địa và chỉ dùng đường băng nội địa.

b) Với hai ràng buộc bổ sung, loại đường băng không cần nằm trong miền giá trị vì đã được xác định bởi biến. Hãy xác định miền mới và vẽ đồ thị ràng buộc của bài toán.

- Vì đường băng đã được ấn định cố định (A,B,C ở Quốc tế; D,E ở Nội địa), biến bây giờ chỉ là khung giờ.

- Miền mới: D_A = D_B = D_C = D_D = D_E = {1, 2, 3, 4}.

- Đồ thị ràng buộc gồm các đỉnh A, B, C, D, E và các cạnh: (A,B), (B,C), (A,C) [do cùng đường băng QT], (D,E) [cùng đường băng NĐ], và cạnh (D,C) [ràng buộc nối chuyến].



c) Sau khi áp dụng arc consistency, miền của các biến còn lại là gì? Bắt đầu bằng việc áp dụng các ràng buộc unary. Hãy gạch bỏ những giá trị bị loại khỏi miền.

- Sau ràng buộc Unary: D_A={1,2}; D_B={1}; D_C={1,2,3,4}; D_D={3,4}; D_E={1,2,3,4}.

- Áp dụng AC-3 (duy trì tính nhất quán cung):

  + A != B và D_B={1} -> loại 1 khỏi A -> D_A={2}.

  + C != B và D_B={1} -> loại 1 khỏi C -> D_C={2,3,4}.

  + C != A và D_A={2} -> loại 2 khỏi C -> D_C={3,4}.

  + D < C: Nếu D=4, không có giá trị nào của C lớn hơn 4 -> loại 4 khỏi D -> D_D={3}. Kéo theo C bắt buộc phải là 4 -> D_C={4}.

  + E != D và D_D={3} -> loại 3 khỏi E -> D_E={1,2,4}.

- Miền còn lại: A={2}, B={1}, C={4}, D={3}, E={1, 2, 4}.

d) Thay vì duy trì arc consistency toàn cục, hãy dùng Forward Checking kết hợp Backtracking Search. Chọn biến theo MRV; khi hòa, chọn ký tự nhỏ hơn trước; chọn giá trị nhỏ hơn trước. Hãy liệt kê các cặp (biến, giá trị) theo đúng thứ tự phát sinh, kể cả các phép gán bị hoàn tác khi gặp dead end. Áp dụng ràng buộc unary trước khi bắt đầu.

A	1	2	3	4
B	1	2	3	4
C	1	2	3	4
D	1	2	3	4
E	1	2	3	4

Áp dụng unary đầu tiên: A={1,2}, B={1}, C={1,2,3,4}, D={3,4}, E={1,2,3,4}.

- B=1 (Do MRV B có 1 giá trị. FC: A={2}, C={2,3,4}).

- A=2 (Do MRV A có 1 giá trị. FC: C={3,4}).

- C=3 (Do MRV C, D có 2 giá trị, ưu tiên chữ cái nhỏ là C. LCV ưu tiên giá trị nhỏ là 3. FC: D < 3 mà D={3,4} -> D bị rỗng). -> Gặp DEAD END.

- Hoàn tác C=3.

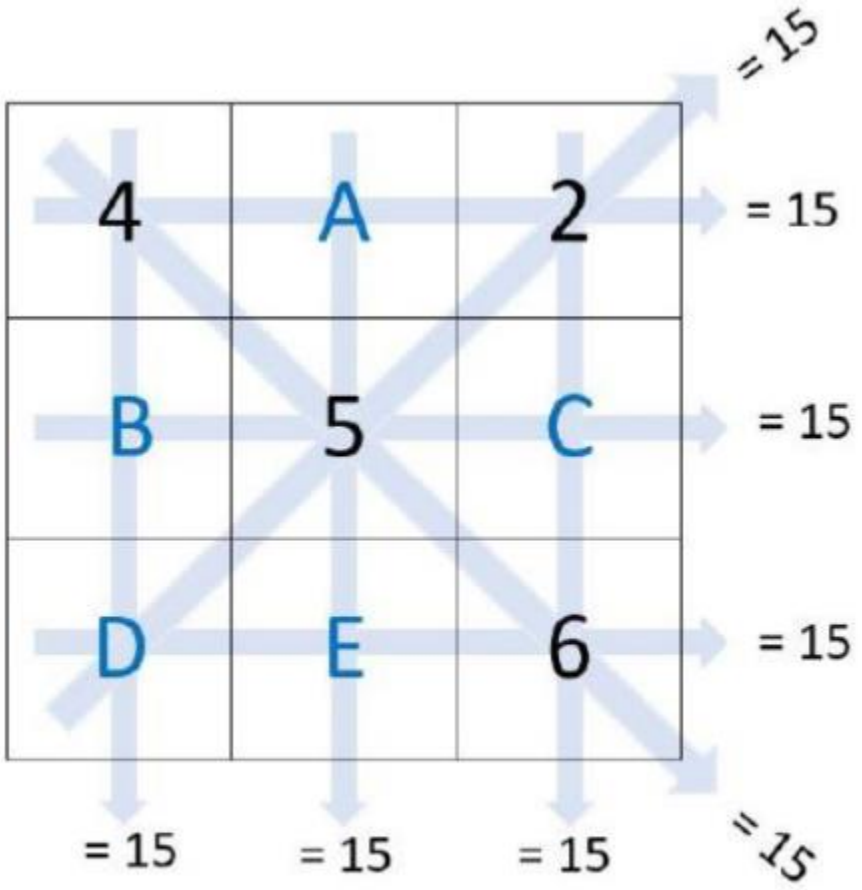
- C=4 (Chuyển sang giá trị tiếp theo. FC: D < 4 -> D={3}).

- D=3 (Do MRV D có 1 giá trị. FC: loại 3 khỏi E -> E={1,2,4}).

- E=1 (Do MRV E, chọn giá trị nhỏ nhất).

Thứ tự phát sinh: B=1 -> A=2 -> C=3 (hoàn tác) -> C=4 -> D=3 -> E=1.

Bài 7. CSP - Magic Square



Hình 6. Bài toán Magic Square 3 × 3

a) Hãy mô hình hóa bài toán dưới dạng CSP: biến, miền giá trị và các ràng buộc.

- Biến: A, B, C, D, E.

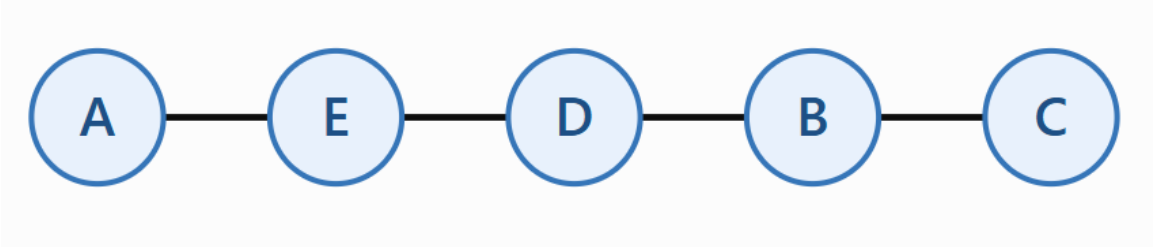
- Miền giá trị: {1, 3, 7, 8, 9} (các số từ 1-9 loại trừ 2, 4, 5, 6 đã điền sẵn).

- Ràng buộc:

+ Ràng buộc tổng = 15 cho các hàng, cột, đường chéo chứa biến: 4+A+2=15; B+5+C=15; D+E+6=15; 4+B+D=15; A+5+E=15; 2+C+6=15; 2+5+D=15.

+ Ràng buộc phân biệt: Alldiff(A, B, C, D, E).

b) Vẽ đồ thị ràng buộc nhị phân.



Rút gọn các biểu thức cộng trừ, ta thu được các ràng buộc Unary: A=9, C=7, D=8. Các ràng buộc Binary: B+C=10, D+E=9, B+D=11, A+E=10. Đồ thị có 5 đỉnh (A,B,C,D,E) liên kết bởi các cạnh: (B,C), (D,E), (B,D), (A,E).

c) Sử dụng đồ thị ràng buộc nhị phân và thuật toán AC-3 để thu hẹp miền, sau đó tìm một nghiệm cho Magic Square.

- Xử lý Unary: Xác định ngay A=9, C=7, D=8. B và E lúc này là {1,3,7,8,9}.

- Kiểm tra AC-3 trên cạnh (B,C) với C=7: B+7=10 -> B=3.

- Kiểm tra AC-3 trên cạnh (D,E) với D=8: 8+E=9 -> E=1.

- Kiểm tra chéo (B,D) 3+8=11 và (A,E) 9+1=10 đều đúng.

- Nghiệm tìm được: A=9, B=3, C=7, D=8, E=1.

Bài 8. MRV và LCV trong thực hành

a) Dùng MRV để chọn biến và LCV để chọn giá trị. Nếu hòa, phá hòa theo thứ tự cầu vồng. Hỏi thuật toán phải backtrack bao nhiêu lần? Hãy ghi lại thứ tự gán.

- Số lần backtrack: 0 lần.

- Thứ tự gán: Ellyse = Hồng -> Sunny = Đỏ -> AJ = Vàng.

- Chi tiết: MRV ưu tiên Ellyse vì chỉ có 2 giá trị. LCV xét Ellyse: chọn "Hồng" chỉ triệt tiêu 1 giá trị của Sunny, trong khi "Đỏ" triệt tiêu 2 giá trị (của Sunny và AJ) -> Chọn Hồng. Tiếp theo MRV xét Sunny (còn 2 giá trị Đỏ, Vàng). Cả hai đều triệt tiêu 1 giá trị của AJ, theo thứ tự cầu vồng ưu tiên "Đỏ" -> Sunny = Đỏ. Cuối cùng, AJ chọn Vàng.

b) Thay MRV bằng Maximum Remaining Values và LCV bằng Most Constraining Value. Hỏi số lần backtrack là bao nhiêu? So sánh với câu (a) và giải thích vì sao.

- Số lần backtrack: 1 lần.

- Giải thích: Heuristic này làm mọi thứ ngược lại. MaxRV sẽ chọn AJ đầu tiên vì có 3 giá trị. MCV chọn giá trị gây hẹp miền nhất là "Đỏ" (triệt tiêu 2 lựa chọn). Sau đó, MaxRV chọn Sunny và MCV gán Sunny="Hồng" (để triệt tiêu lựa chọn còn lại duy nhất của

Ellyse). Điều này làm Ellyse hết giá trị -> bế tắc (Dead end) và bắt buộc phải backtrack lại Sunny để gán giá trị Vàng.

- So sánh: Trong khi MRV/LCV xử lý êm xuôi không cần backtrack do áp dụng chiến lược thông minh (giải quyết biến khó trước, giữ lựa chọn mở), thì MaxRV/MCV đi vào con đường "tự hủy", làm khó các biến sau khiến thuật toán mất thời gian quay lui.